

DYNAMIC REQUEST BATCHING

with Deep Reinforcement Learning

Shirish Giroti

SAC

Soft Actor-Critic

Off-policy RL with entropy bonus

LSTM

Memory Network

300 ms context window

PER

Prioritized Replay

Focus on surprising transitions

The Problem

Deciding when to dispatch a batch of requests

Modern servers receive thousands of requests per second.

Dispatch too early → tiny batches, wasted overhead

Dispatch too late → SLA violation (>500 ms)

Traffic varies between peak hours and quiet hours

GOAL

Train an RL agent that learns the optimal batching policy from experience alone
— adaptive across traffic conditions, with zero SLA violations.

Request Flow



Incoming requests



QUEUE (size grows over time)



WAIT or
SERVE?

(Agent decides every 10 ms)

Environment: State & Actions

What the agent observes and what it can do

State (8 dimensions)

1. Queue length (normalised)
2. Arrival rate EMA (req/s)
3. Time since last dispatch (ms)
4. Oldest request wait time (ms)
5. Mean request wait time (ms)
6. Rate delta (rate change)
7. Time of day (0–1)
8. SLA headroom (ms)

Actions (4 discrete choices)

W0	Serve immediately <i>Dispatch the queue right now</i>
W20	Wait 20 ms <i>Accumulate 20 ms of arrivals</i>
W50	Wait 50 ms <i>Accumulate 50 ms of arrivals</i>
W100	Wait 100 ms <i>Maximum batch accumulation</i>

Reward = +batch_size - empty_serve_penalty - SLA_violation

Rewards large batches, penalises empty dispatches and missed deadlines

The Algorithm: Three Components

SAC + LSTM + PER working together

SAC

The decision-maker

Soft Actor-Critic

Maintains an actor (policy) and twin critics (Q-values). Uses entropy regularisation to keep exploring rather than committing too early. Alpha curriculum anneals from 1.0 to 0.1 over 100 episodes.

- Off-policy: reuses any past experience
- Twin critics combat overestimation bias
- Target networks via Polyak averaging ($\tau=0.005$)

LSTM

The temporal context

Memory Network

Looks at the last 30 states (300 ms of history) to detect burst onsets before the queue visibly grows. Replaces stateless feed-forward processing.

- 2-layer LSTM, 128 hidden units, 10% dropout
- LayerNorm prevents gradient explosion
- Dueling critic: $Q(s,a) = V(s) + A(s,a) - \text{mean}(A)$

PER

The memory manager

Prioritized Replay

Samples surprising transitions (high TD error) more often than routine ones. Importance-sampling weights correct the bias from non-uniform sampling.

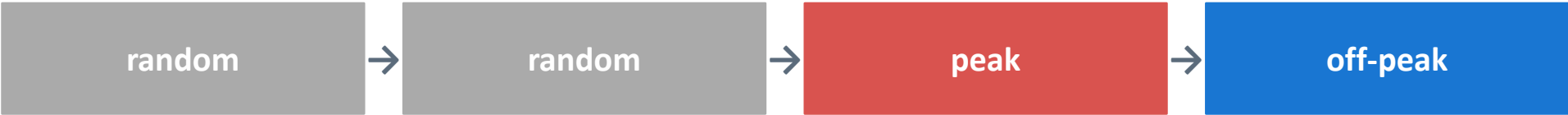
- Buffer capacity: 200,000 transitions
- Priority = $(|\text{TD-error}| + \epsilon)^{0.6}$
- Beta anneals 0.4 $\rightarrow$ 1.0 over 200k steps

Training Setup

Three traffic patterns, balanced phase cycle



Phase cycle: random → random → peak → off-peak (repeated 125 times)



Three Traffic Patterns

Pattern	Peak rate	Off-peak rate	Character
Bursty	500 req/s	20 req/s	Extreme bimodal
Poisson	120 req/s	80 req/s	Mild variation
Time-varying	250 req/s	50 req/s	Realistic

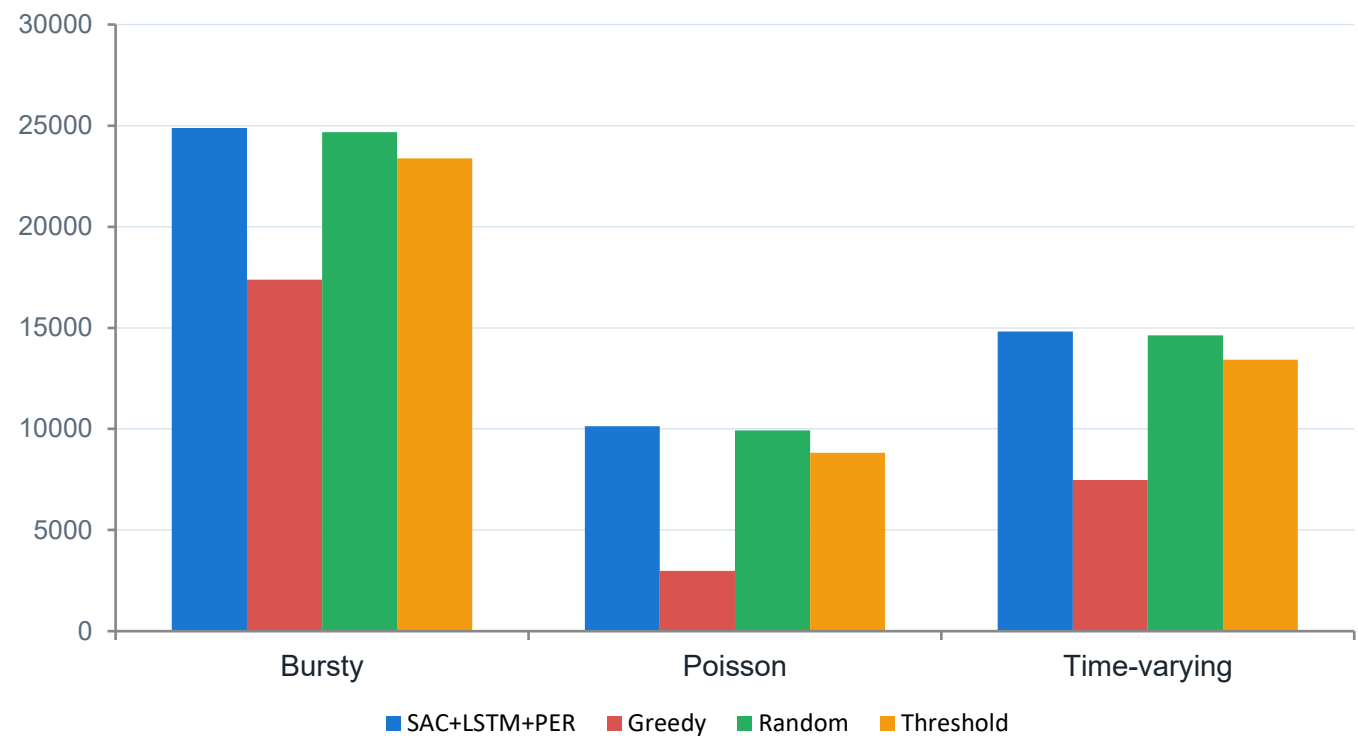
Key Hyperparameters

Parameter	Value
State dimension	8
Sequence length	30 (300 ms)
Discount γ	0.99
Learning rate	3×10^{-4}
Buffer capacity	200,000
Alpha curriculum	1.0 → 0.1

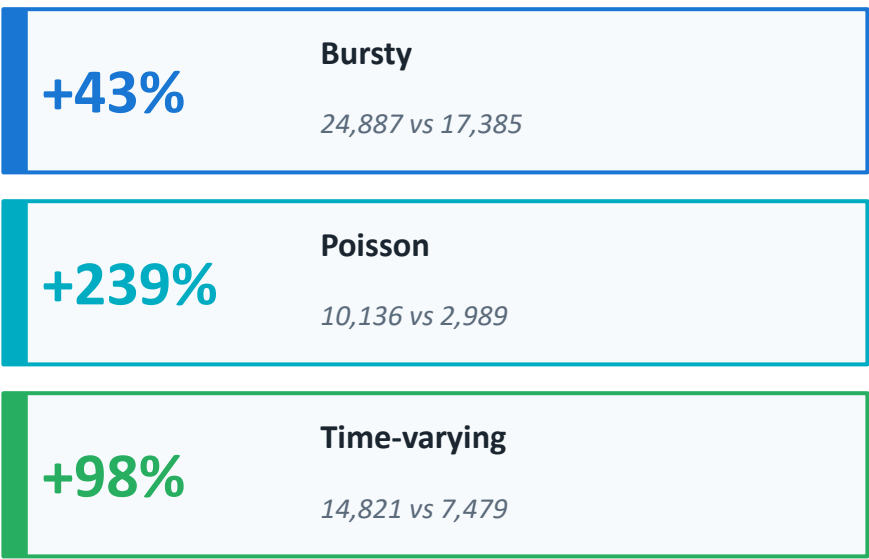
Results: SAC vs. Baselines

Stratified evaluation — 100 peak + 100 off-peak episodes per pattern

Overall Mean Reward (across all 3 traffic patterns)



SAC Advantage vs Greedy



Zero SLA violations across all 600 evaluation episodes — SAC achieves maximum batch efficiency with no latency cost.

What the Agent Learned

Convergence to the mathematically optimal policy

Final policy

W100 = 100%

Always wait 100 ms before serving

Discovered through trial-and-error within 50 episodes. No domain knowledge was provided to the agent — it found the optimal policy by exploring all 4 actions and observing rewards.

Why W100 dominates

Action	Bursty Peak (500 req/s)	Reward / action
W0 (serve)	~1 req	~1.5
W20	~10 req	~15
W50	~25 req	~38
W100	~50 req	~75

Math says:

100 ms × 500 req/s = 50 requests per batch.
Serve immediately = 1 request per batch.
→ 50× batch efficiency gain.

The agent learned an interpretable, provably optimal policy from raw experience — the central success of this project.

Conclusion

1

Optimal policy discovered

SAC+LSTM+PER learned to always wait 100 ms and batch ~50 requests per dispatch — a 50× efficiency gain over greedy serving.

2

Wins on all 3 traffic types

+43% on bursty, +239% on Poisson, +98% on time-varying. The largest gain on Poisson proves the advantage is general, not just burst-specific.

3

Zero SLA violations

Across all 600 evaluation episodes, no request ever exceeded the 500 ms deadline. Maximum efficiency at zero latency cost.

Thank you

Questions welcome